Dynamic Monitoring of Radiological Pollution through Deep Learning

Classifying anomaly shapes in noisy gamma-dose time series

Pedro De Bem
[co-authors — to be completed]

CentraleSupélec — Pôle Projet Data Science
Supervised by Elisabeth Lahalle (L2S, CentraleSupélec)

June 2026

Abstract

Environmental beacons monitor radiological pollution by counting gamma photons reaching a scintillation detector. The resulting time series is heavily noisy and slowly drifting, and the events of interest — spikes, plateaus, ramps — are rare and vary in shape, amplitude and duration. We formalise the task as *sliding-window multi-class classification* of a univariate gamma-dose signal, and we validate a deep-learning approach to it on a controlled synthetic benchmark. Because no labelled real events are available, we generate a deliberately simplified signal model — white Gaussian high-frequency noise on an Ornstein–Uhlenbeck baseline, calibrated to the statistics of four months of real 2015 recordings, with six injected anomaly morphologies. On this benchmark we train and compare three architectures of increasing complexity: a compact 1D-CNN, a 1D residual network, and a hybrid CNN with a Transformer encoder. All three reach a strong event-level performance (macro F_1 from 0.93 to 1.00 on a held-out segment of 59 events); the residual network is the best overall, combining a perfect event-level score with by far the highest robustness to background-noise growth. We further compare the three on robustness and computational cost. Evaluating the trained networks on real recordings is deliberately out of scope: the simplified models serve only to validate the approach, and bridging the synthetic-to-real gap is left as the main perspective.

Keywords: time-series classification, 1D convolutional networks, residual networks, self-attention, radiological monitoring, synthetic data generation, anomaly detection.

Contents

List of Figures

Chapter 1

Introduction

1.1 Context

Environmental surveillance beacons monitor radiological pollution by counting, at a fixed sampling interval (one minute here), the gamma photons that reach a scintillation detector. The resulting *gamma-dose* time series reflects the natural radioactive background, modulated by weather and, rarely, by a radiological event: the passage of a contaminated source, a leak, or the dispersal of a radioactive plume. The operational goal is to detect and *identify* such events as early as possible, directly from the shape they leave in the signal. Three properties make this harder than textbook anomaly detection: the per-sample noise is comparable in size to weak events; the baseline drifts slowly with meteorological conditions, so no fixed threshold works; and different physical phenomena leave *different* temporal signatures, so detection alone is not enough — the response must classify the shape.

1.2 Problem formalisation

We treat the problem as supervised classification on a univariate signal. Let $x = (x_1, \dots, x_N) \in \mathbb{R}^N$ be the gamma-dose series sampled at uniform intervals. Each instant t belongs to one of K classes: a `Normal Background` class (label 0) and $K-1$ anomaly morphologies. The instantaneous value x_t carries almost no class information — a class is defined by the local *shape* of the signal — so we classify fixed-length windows rather than samples.

Windowing. For a window length W we map each position t to the sub-sequence

$$\mathbf{x}^{(t)} = (x_{t-W+1},\, \dots,\, x_t) \in \mathbb{R}^W, \tag{1.1}$$

and ask a model $f_\theta : \mathbb{R}^W \to \mathbb{R}^K$, with parameters θ, to map it to class logits whose softmax $\hat{p}^{(t)} = \mathrm{softmax}\big(f_\theta(\mathbf{x}^{(t)})\big) \in \Delta^{K-1}$ is a distribution over the K classes.[1] We use $W = 512$ samples (≈ 8.5 hours). Each window receives a single label $y^{(t)} \in \{0, \dots, K-1\}$ through the coverage rule of section 4.1.

Learning objective. Given a training set $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}$ of labelled windows, θ is fitted by minimising the class-weighted cross-entropy

$$\mathcal{L}(\theta) = -\frac{1}{|\mathcal{D}|} \sum_{(\mathbf{x},y)\in\mathcal{D}} w_y \, \log \mathrm{softmax}\big(f_\theta(\mathbf{x})\big)_y, \qquad w_c = \frac{|\mathcal{D}|}{K\, n_c}, \tag{1.2}$$

[1] Δ^{K-1} is the probability simplex, i.e. the set of non-negative vectors of $\mathbb{R}^K$ summing to 1.

where n_c is the number of training windows of class c and the weight w_c compensates the strong class imbalance (section 4.4).

Evaluation objective. Sample- or window-level accuracy over-counts the many background windows and the fuzzy boundaries of an event. The operationally meaningful quantity is whether each *event* is detected and correctly named. We therefore also report an *event-level* macro F_1-score (section 5.2): predictions are aggregated into a per-sample confidence trace, contiguous non-background runs form predicted events, and each is matched against the ground-truth events. This bridges window classification and the detection task the beacon actually performs.

1.3 Why deep learning, and what we validate

A classical pipeline — baseline removal followed by a bank of hand-tuned matched filters — works on clean, isolated shapes but scales poorly: real events are time-warped and noisy, cross-correlation with a template degrades quickly at the observed noise level, and every new morphology requires redesigning the bank. Convolutional and attention-based networks instead *learn* their own shape features from data and are, by now, a strong baseline for time-series classification (chapter 2). The project brief [10] explicitly suggests evaluating convolutional and attention layers, which is what we do.

Crucially, **no labelled real events are available**. Following the methodological choice that a deliberately simplified, well-understood synthetic model is the right tool to *validate an approach*, we generate a synthetic benchmark whose noise statistics are calibrated to real recordings, and we evaluate the networks only on held-out *synthetic* data. Running networks trained on a simplified model against real signals would measure the modelling gap, not the classifier, and is deliberately left as future work (chapter 6).

1.4 Contributions and outline

This report (i) formalises radiological-event identification as windowed multi-class classification (section 1.2); (ii) positions the work against the deep-learning time-series literature (chapter 2); (iii) describes a simplified, statistically calibrated synthetic gamma-dose generator (chapter 3); (iv) specifies three architectures of increasing complexity and a shared training and event-evaluation pipeline (chapter 4); and (v) compares them on classification performance, robustness and computational cost (chapter 5). Chapter 6 concludes and lays out the path towards more realistic signal models and real-data evaluation.

Chapter 2

Related work and positioning

This chapter situates our work in the deep-learning literature for time series, limited to the ideas we actually build on, and states our positioning at the end.

2.1 Deep learning for time-series classification

Time-series classification (TSC) has shifted from distance-based methods (e.g. nearest neighbour with dynamic time warping) to deep learning. The review of Ismail Fawaz et al. [8] benchmarks the main families and identifies fully-convolutional and residual networks as the strongest general-purpose baselines, a conclusion already drawn by Wang et al. [15], whose end-to-end fully-convolutional network (FCN) and 1D ResNet outperform most hand-crafted pipelines without feature engineering. These two works directly motivate our choice of a convolutional baseline and a residual variant. Temporal convolutional networks [2] show, more generally, that convolutions with a sufficient receptive field match or beat recurrent models on sequence tasks while being cheaper to train — relevant to an operational beacon with a limited compute budget.

2.2 Convolutional and residual architectures

The convolutional building block goes back to LeCun et al. [11]: weight-shared local filters give translation equivariance and a parameter count independent of the input length, which is exactly what is needed to detect an anomaly that may appear anywhere in a window. Very deep stacks are hard to optimise because gradients vanish; He et al. [5] solve this with *residual* connections that let a block default to the identity, making depth essentially free. Training is further stabilised by normalisation: batch normalisation [7] reduces internal covariate shift but couples train and inference statistics, whereas group normalisation [16] computes statistics per sample and so behaves identically at train and inference time — a property we exploit for a signal whose amplitude scale drifts.

2.3 Attention and Transformer encoders

Self-attention [14] lets every position in a sequence attend to every other with a learnt, content-dependent weighting, giving an unbounded receptive field in a single layer; layer normalisation [1] and a learnable aggregation token [4] make such encoders practical for classification. Hybrids that place a Transformer encoder on top of a convolutional stem are now common for noisy physiological signals: Hu and Chen [6], cited in the project brief, combine convolutions with

multi-head self-attention for EEG emotion recognition. Our third architecture follows the same recipe, transposed to gamma-dose signals.

2.4 Anomaly detection and explainability

Our task is closer to *classification of known morphologies* than to unsupervised anomaly detection, the latter surveyed by Blázquez-García et al. [3]; we borrow from it the event-level, detection-oriented evaluation rather than a purely point-wise accuracy. Finally, because an operational alert must be trustworthy, explainability matters: the brief points to Grad-CAM [13], which produces gradient-based saliency over the input. Our convolutional models expose the hooks required for it (chapter 6), but a full explainability study is out of scope here.

2.5 Positioning

We adopt established architectures — a convolutional baseline, a residual network [5, 15], and a CNN–Transformer hybrid [14, 6] — rather than proposing a new one. The specific contribution of this project is therefore not architectural but methodological and applicative: (i) a *simplified yet statistically calibrated* synthetic generator for noisy gamma-dose signals with controllable anomaly morphologies, and (ii) a controlled, like-for-like comparison of the three architectures along the three axes named in the brief — classification performance, computational cost, and robustness to distribution shift. To our knowledge this combination has not been reported for radiological-beacon anomaly classification.

Chapter 3

Signal model and synthetic data

No labelled real events are available, so the networks are trained and evaluated on synthetic data. This chapter first characterises the real recordings just enough to fix realistic parameters (section 3.1), then defines the background model (section 3.2), the anomaly morphologies (section 3.3) and the generator (section 3.4). The model is *deliberately simplified*: its role is to validate the classification approach under controlled, well-understood conditions, not to reproduce real signals faithfully.

3.1 Real data: a brief characterisation

The reference data are four months of 2015 gamma-dose recordings provided by the supervisor (Feb, Apr, Jun, Oct), each $\approx$ 40,000 samples at one sample per minute. Figure 3.1 shows a representative month. Two components are visible: a slowly varying baseline around $\sim$ 99 counts, and a high-frequency, zero-mean fluctuation of a few counts. Separating the two with a moving-average baseline b_t and residual $r_t = x_t - b_t$ gives the per-month statistics of table 3.1: the baseline sits near 99 counts and the high-frequency residual has standard deviation $\sigma \approx 2.3$–2.7. These few numbers are all we take from the real data.

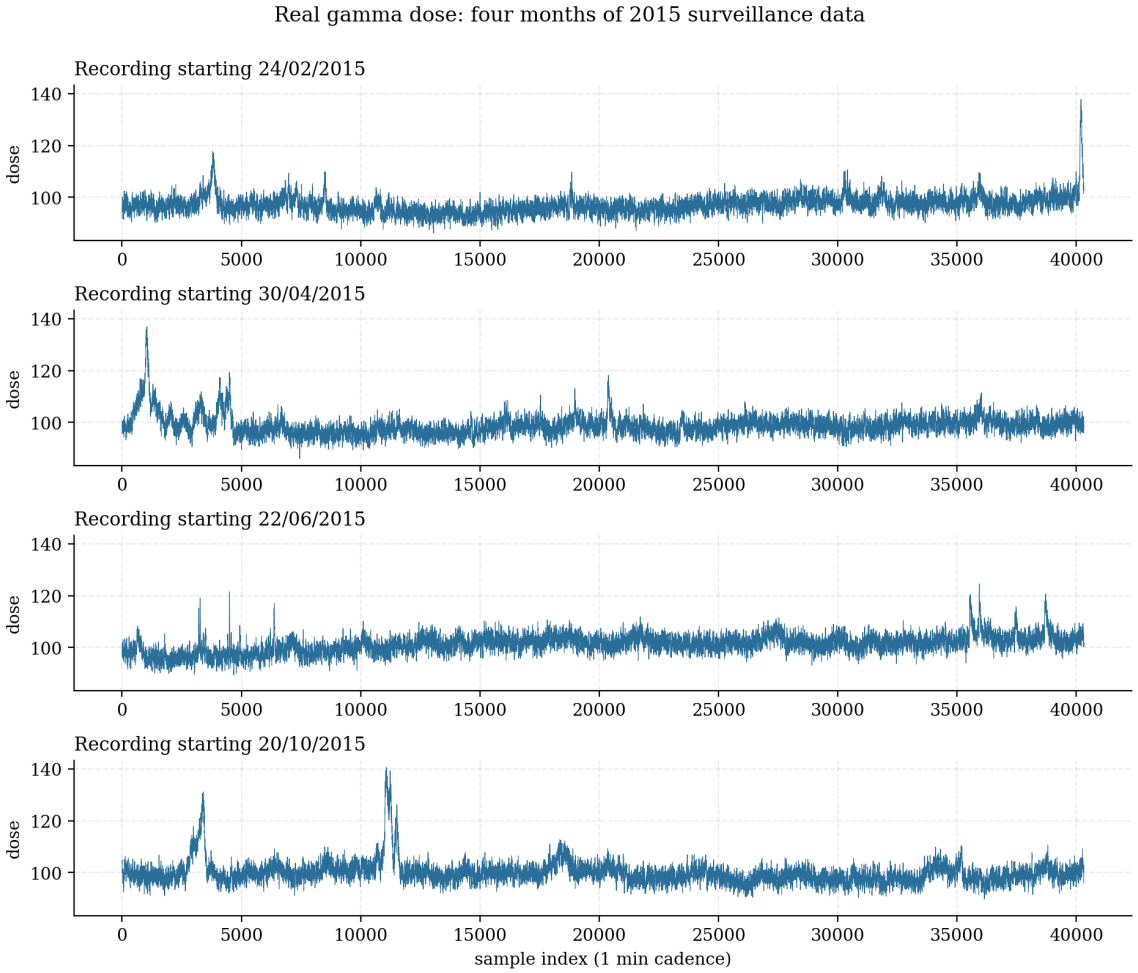

Figure 3.1. The four months of real 2015 gamma-dose recordings (one sample per minute). The signal is a slowly drifting baseline near 99 counts plus a few-count high-frequency fluctuation, with occasional sharper excursions.

Month	Mean μ	HF noise std σ	OU θ	OU σ_{OU}
24/02/2015	96.93	2.31	$4 \cdot 10^{-6}$	0.0277
30/04/2015	98.86	2.58	$4 \cdot 10^{-6}$	0.0390
22/06/2015	101.28	2.51	$4 \cdot 10^{-6}$	0.0337
20/10/2015	99.95	2.70	$4 \cdot 10^{-6}$	0.0485
Average	**99.26**	**2.53**	$\mathbf{4 \cdot 10^{-6}}$	**0.0373**

Table 3.1. Statistics of the four real months. The average row gives the parameters used by the synthetic generator.

3.2 Background model

Following section 3.1, the synthetic background is a white Gaussian high-frequency noise on a slowly drifting baseline:

$$x_t = b_t + \eta_t, \quad \eta_t \overset{\text{iid}}{\sim} \mathcal{N}(0, \sigma^2), \quad b_{t+1} = b_t + \theta\,(\mu - b_t) + \xi_t, \quad \xi_t \overset{\text{iid}}{\sim} \mathcal{N}(0, \sigma_{\mathrm{OU}}^2), \tag{3.1}$$

with $\mu = 99.26$, $\sigma = 2.53$, $\theta = 4 \cdot 10^{-6}$ and $\sigma_{\mathrm{OU}} = 0.0373$ (table 3.1). The first equation makes each reading a baseline plus zero-mean Gaussian sensor noise; the second is a discrete

Ornstein–Uhlenbeck (OU) process, a mean-reverting random walk whose very small θ keeps the baseline only loosely tethered to μ, so it drifts over hundreds of thousands of samples — much slower than the $W = 512$ analysis window. This model is intentionally crude: it captures the marginal noise level and slow drift, but treats the high-frequency noise as independent in time, which is the main simplification we accept at this stage (chapter 6).

3.3 Anomaly morphologies

We model six anomaly morphologies, giving $K = 7$ classes with the background. Each is stored as a small set of *keypoints* (normalised time and amplitude) joined by linear, exponential or smoothed half-cosine segments (fig. 3.2): `bell` and `bell_sq` (smooth bumps, the second with a short plateau), `fast_ascend` and `fast_descend` (asymmetric ramps with a fast edge and a slow tail), `M` (a twin-peak episode) and `square` (a sustained plateau).

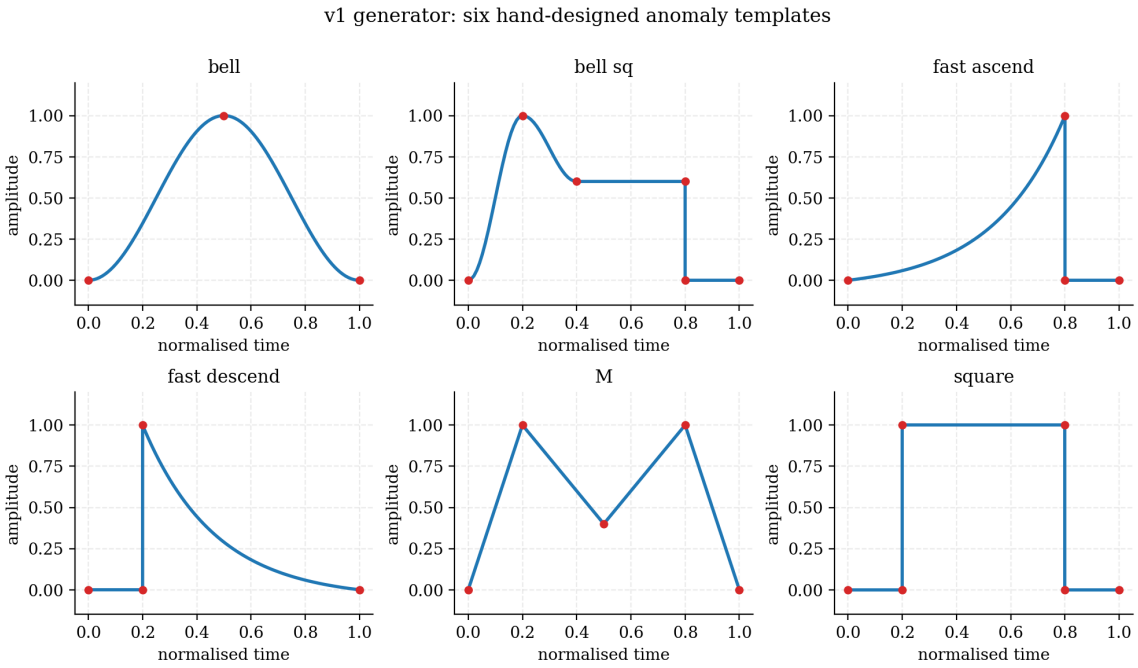

Figure 3.2. The six anomaly templates in normalised time and amplitude. Each is a short list of keypoints (markers) connected by linear, exponential or half-cosine segments.

A template used verbatim would not generalise, so every injected anomaly is randomly deformed before use (fig. 3.3). Each keypoint is jittered by a Gaussian of standard deviation ν (the *deformation level*) under direction constraints that preserve the qualitative shape, time monotonicity is enforced, and the curve is then rescaled to a target amplitude and duration. The generator samples

$$\text{amplitude} \in \sigma \cdot [2.5,\ 7.6], \qquad \text{duration} \in [200,\ 800] \text{ samples}, \qquad \nu \in [0.02,\ 0.10]. \tag{3.2}$$

Amplitudes are expressed in multiples of the noise std σ, so the per-sample signal-to-noise ratio of an anomaly lies in the interesting range $[2.5, 7.6]$ — strong enough to be learnable, weak enough to be non-trivial.

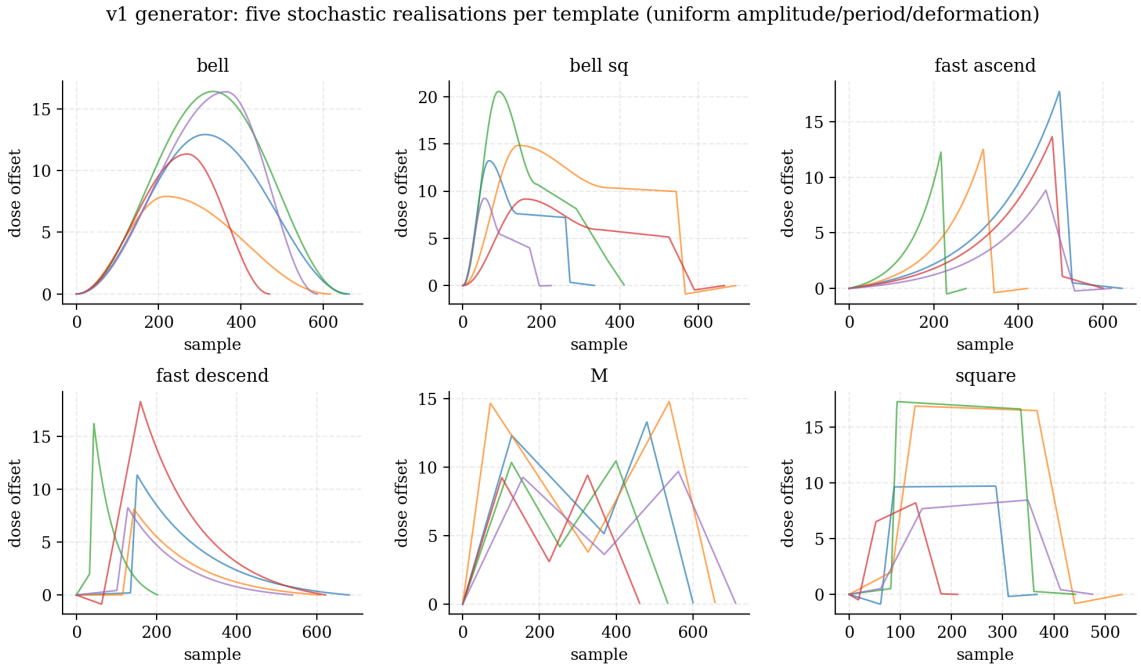

Figure 3.3. Five random realisations of each template under the ranges of eq. (3.2). The qualitative shape is preserved while amplitude, duration and fine shape vary.

3.4 The synthetic dataset

The generator combines eqs. (3.1) and (3.2) into one long labelled signal. It (i) draws a number of anomalies and their roughly evenly spaced injection points (with ±200-sample jitter); (ii) for each, samples a template and a (amplitude, duration, ν) triple and builds the deformed curve; (iii) generates the background of eq. (3.1) in 50,000-sample chunks (so memory stays constant); and (iv) adds each anomaly on top of the background and labels every sample with its class. Figure 3.4 shows two injected anomalies up close, each overlaid with its underlying clean shape.

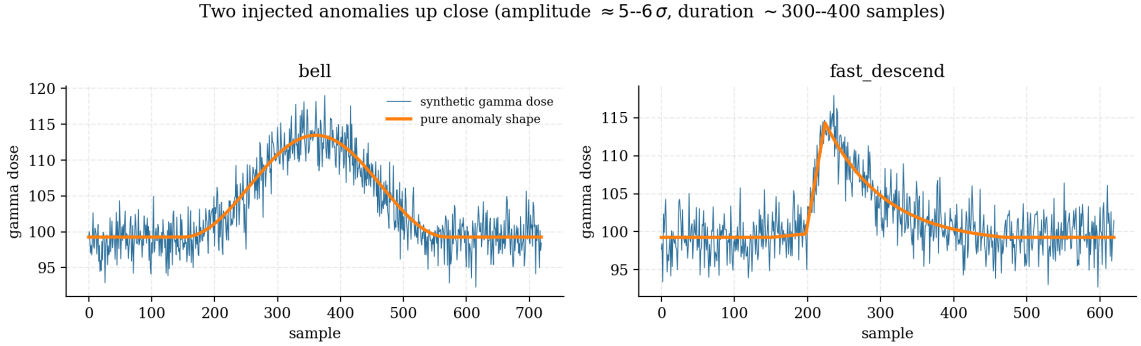

Figure 3.4. Two injected anomalies seen in the synthetic signal. Blue: the generated gamma dose (baseline + Gaussian noise + anomaly); orange: the pure anomaly shape before noise. Left: a smooth `bell`; right: an asymmetric `fast_descend` (fast rise, slow decay). Both have an amplitude of ~ 5–$6\,\sigma$ above the baseline.

The dataset used throughout this report has $N = 10^7$ samples with 5,000–8,000 injected anomalies, i.e. a density of $\sim$ 6–8 events per 10,000 samples (about one anomaly every 1,500 samples). This dense injection is a deliberate choice for the validation benchmark: it provides abundant, balanced examples of every morphology, at the cost of a background prior that is far denser than

a real beacon would see — a gap we return to in chapter 6.

Chapter 4

Methods: architectures and training

We compare three architectures of increasing complexity. They share the same interface — input a single-channel window $\mathbf{x} \in \mathbb{R}^{1\times 512}$, output $K = 7$ class logits — so they are interchangeable in one training and evaluation pipeline. This chapter covers the input pipeline (section 4.1), the building blocks we use (section 4.2), the three networks (section 4.3) and the training recipe (section 4.4). We describe only the operations the models actually use.

4.1 Windowing, labelling and normalisation

Temporal split and windows. To avoid leakage between overlapping windows, the raw signal is split along time first (first 80% for train + validation, last 20% for test); sliding windows of length $W = 512$ are then extracted independently in each part with stride 32 at training time.

Window labelling. A window may straddle background and an event. We label it by *coverage*: if anomalous samples make up at least $\tau_{\text{cov}} = 10\%$ of the window, the window takes the majority anomaly class among them; otherwise it is `Normal` (fig. 4.1). The 10% floor keeps a window from being called anomalous on the strength of a few boundary samples, while the abundant injection density of the benchmark (section 3.4) still yields many positive windows per morphology.

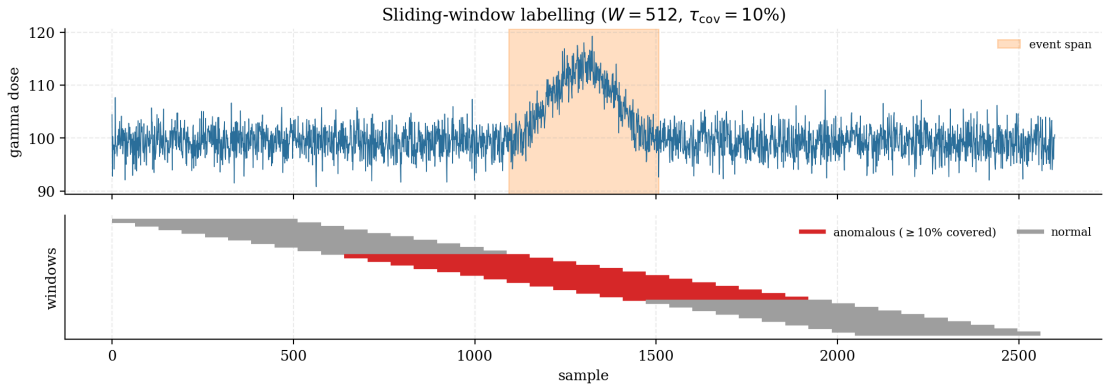

Figure 4.1. Sliding-window labelling on a fragment containing one event (orange span, top). Each bar (bottom) is one 512-sample window; a window is labelled anomalous (red) when at least $\tau_{\text{cov}} = 10\%$ of its samples fall inside the event, and `Normal` (grey) otherwise.

Per-window normalisation. Because the baseline drifts, each window is centred on its own mean and divided by a single global standard deviation σ_{train} estimated on the centred training

windows:

$$\tilde{\mathbf{x}}^{(t)} = \frac{\mathbf{x}^{(t)} - \overline{\mathbf{x}^{(t)}}}{\sigma_{\text{train}}}, \qquad \sigma_{\text{train}} = 4.21. \tag{4.1}$$

The network thus sees only deviations from the local baseline, and the same σ_{train} is reused at inference.

4.2 Building blocks

1D convolution. The core operator maps a multi-channel sequence $\mathbf{u} \in \mathbb{R}^{C_{\text{in}} \times L}$ to $\mathbf{v} \in \mathbb{R}^{C_{\text{out}} \times L'}$ through a learnable kernel of width k:

$$\mathbf{v}_{c,t} = b_c + \sum_{c'=1}^{C_{\text{in}}} \sum_{i=1}^{k} \mathbf{w}_{c,c',i}\ \mathbf{u}_{c',\, s\,t+i-p}, \tag{4.2}$$

with stride s and padding p. Weights are shared across time, giving translation equivariance (an anomaly is recognised wherever it occurs) and a parameter count $C_{\text{out}}C_{\text{in}}k$ independent of L. Each output channel acts as a learned detector of a local shape. Between convolutions we apply the pointwise nonlinearity $\mathrm{ReLU}(z) = \max(0, z)$ and reduce resolution with max-pooling; an `AdaptiveAvgPool1d(4)` near the end keeps four temporal bins, retaining a coarse "where in the window" cue useful to separate, e.g., `fast_ascend` from `fast_descend`.

Normalisation. Batch normalisation [7] standardises each channel over the mini-batch; it is effective but uses running statistics that differ between train and inference. Group normalisation [16] instead normalises over groups of channels *per sample*, so it behaves identically at train and inference and is amplitude-invariant per window — a good match for eq. (4.1). We use batch norm in the baseline and group norm (8 groups) in the two deeper models.

Residual block. To train depth without vanishing gradients, a residual block [5] learns a correction to the identity,

$$\mathrm{Res}(\mathbf{u}) = \mathrm{ReLU}\big(\mathbf{u} + \mathcal{F}(\mathbf{u})\big), \tag{4.3}$$

where $\mathcal{F}$ is a small Conv–GroupNorm–ReLU stack and a 1×1 convolution adapts the shortcut when channel counts change.

Self-attention. For the hybrid model we use a Transformer encoder [14]. Given a token sequence $\mathbf{Z} \in \mathbb{R}^{T \times d}$, attention forms queries, keys and values $\mathbf{Q}, \mathbf{K}, \mathbf{V} = \mathbf{Z}\mathbf{W}^Q, \mathbf{Z}\mathbf{W}^K, \mathbf{Z}\mathbf{W}^V$ and mixes positions by content:

$$\mathrm{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \mathrm{softmax}\Big(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_h}}\Big)\,\mathbf{V}. \tag{4.4}$$

Several such heads run in parallel (multi-head attention). Unlike a convolution, one attention layer gives every position access to the whole window, with a learnt weighting. Since eq. (4.4) is order-agnostic, a fixed sinusoidal positional encoding is added to the tokens; a learnable `[CLS]` token [4], which attends to all others, is read out for classification.

4.3 The three architectures

A — 1D-CNN (baseline). Three Conv–BatchNorm–ReLU–MaxPool blocks (kernels $7, 7, 5$; channels $16 \to 32 \to 64$), then `AdaptiveAvgPool1d(4)` and an MLP head (table 4.1). It is the simplest model and a sanity check on the pipeline.

Block	Operation	Output
Input	raw window	$(1, 512)$
Block 1	Conv1D($1 \to 16, k{=}7$) $\to$ BN $\to$ ReLU $\to$ MaxPool(2)	$(16, 256)$
Block 2	Conv1D($16 \to 32, k{=}7$) $\to$ BN $\to$ ReLU $\to$ MaxPool(2)	$(32, 128)$
Block 3	Conv1D($32 \to 64, k{=}5$) $\to$ BN $\to$ ReLU $\to$ MaxPool(2)	$(64, 64)$
Pool	AdaptiveAvgPool1d(4)	$(64, 4)$
Head	Flatten $\to$ Drop(0.4) $\to$ Lin($256 \to 32$) $\to$ ReLU $\to$ Drop(0.2) $\to$ Lin($32 \to K$)	(K)

Table 4.1. Architecture A — 1D-CNN (22,727 parameters).

B — ResNet 1D. The same depth, but each block is a residual block (eq. (4.3)) with group norm; first-block kernels $(15, 9, 5)$ for broad context, then $(7, 5, 3)$; channels $16 \to 32 \to 64$ with max-pool between blocks, then the same pooling and head as model A. Keeping the head identical makes the residual trunk the only difference from A — a clean ablation.

C — CNN+Attention. A stride-2 convolutional stem (channels $16 \to 32 \to 64$, group norm) reduces the length eightfold to $T = 64$ tokens of dimension $d = 64$. A learnable `[CLS]` token is prepended, a sinusoidal positional encoding added, and two pre-norm Transformer encoder layers ($h = 4$ heads, feed-forward width 256, dropout 0.1) are applied; the `[CLS]` output feeds a linear classifier. The stride-2 stem keeps the $O(T^2)$ attention cost small.

	1D-CNN	ResNet 1D	CNN+Attention
Trunk	3 plain conv blocks	3 residual blocks	stride-2 stem + 2 encoders
Normalisation	BatchNorm	GroupNorm(8)	GroupNorm(8) + LayerNorm
Long-range mixing	—	—	multi-head self-attention
Parameters	22,727	74,631	109,655

Table 4.2. The three architectures at a glance ($K = 7$).

4.4 Training

All models use one recipe (full settings in section A.2), implemented in PyTorch [12]. We minimise the class-weighted cross-entropy of eq. (1.2) with Adam [9] (learning rate 10^{-3}), batch size 128, for at most 35 epochs. The learning rate is halved after two epochs without validation improvement (`ReduceLROnPlateau`), and training stops early after ten such epochs, keeping the lowest-validation-loss weights. The dataset yields 249,985 training windows (split 80/20 into train and validation) and 62,485 test windows. The background class makes up only $\sim 31\%$ of windows and each anomaly ~ 11–12%; the inverse-frequency weights of eq. (1.2) are $\mathbf{w} = (0.46, 1.18, 1.27, 1.32, 1.24, 1.15, 1.31)$. No data augmentation is used, as the stochastic generator already supplies variability.

Chapter 5

Experiments and results

All results are on held-out *synthetic* data: window-level accuracy on the test split (section 5.1), event-level detection on a continuous segment (section 5.2), robustness to distribution shift (section 5.3) and computational cost (section 5.4). Section 5.5 discusses the trade-offs.

5.1 Training dynamics and window-level accuracy

The three models converge within 35 epochs (fig. 5.1). The two deeper models reach a clearly lower validation loss than the baseline (≈ 0.57 for the ResNet and the CNN+Attention vs ≈ 0.72 for the 1D-CNN) and start higher on validation F_1.

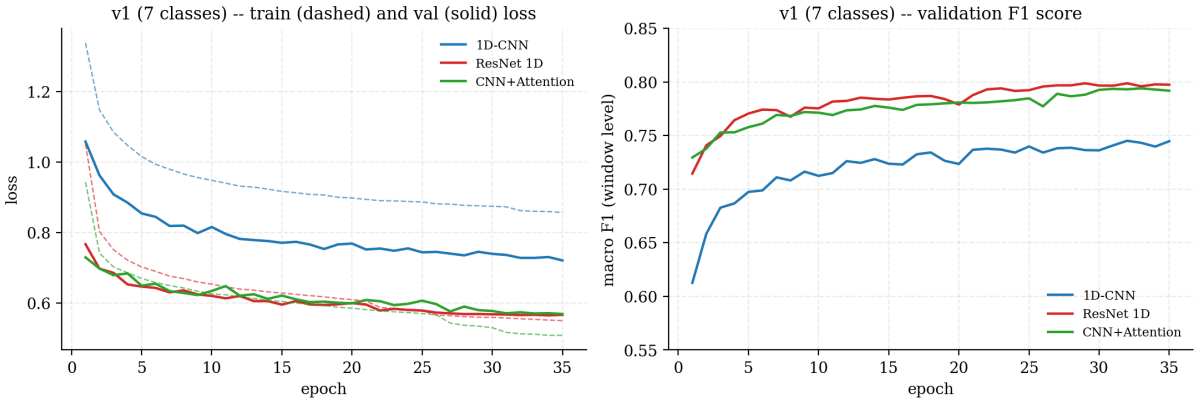

Figure 5.1. Training dynamics. Train/validation loss (left) and validation macro F_1 (right) for the three models.

On the 62,485 held-out test windows, the per-class and overall accuracies are reported in table 5.1. The background class is recognised at $\sim 94\%$ by all models; the anomaly classes are harder and tightly grouped, with `bell_sq` the hardest (it is a `bell` with an added plateau, easily confused with both `bell` and `square`). Depth helps: the ResNet leads overall, the CNN+Attention is close behind, and both clearly beat the baseline.

Class	1D-CNN	ResNet 1D	CNN+Attention
`Normal Background`	0.941	0.942	0.911
`bell`	0.697	0.779	0.765
`bell_sq`	0.619	0.713	0.712
`fast_ascend`	0.704	0.737	0.744
`fast_descend`	0.721	0.741	0.738
`M`	0.687	0.765	0.766
`square`	0.703	0.740	0.732
Overall accuracy	**0.766**	**0.807**	**0.795**

Table 5.1. Window-level accuracy on the held-out test set (62,485 windows).

5.2 Event-level evaluation

Window accuracy understates operational quality: the many background windows inflate it, and boundary windows flip easily. We therefore evaluate detection on a continuous 100,000-sample segment (immediately after the training portion, 59 ground-truth events). The window outputs are averaged into a per-sample probability trace; a sample is anomalous when its background probability is below 0.3; contiguous anomalous runs (gaps up to 50 samples bridged) form predicted events, each named by the arg-max of its cumulative probability. A predicted event matches a true event when their intervals overlap, splitting results into hits, misses and false alarms, from which we compute precision, recall and a macro-averaged F_1 over the anomaly classes.

Figure 5.2 shows the event-level confusion matrices and table 5.2 the summary. The picture is sharp: no model raises a single false alarm on the segment, and the ranking from table 5.1 is amplified. The **ResNet classifies all** 59 **events correctly** (macro $F_1 = 1.00$); the CNN+Attention misses 3 and the 1D-CNN misses 4, both still above 0.93.

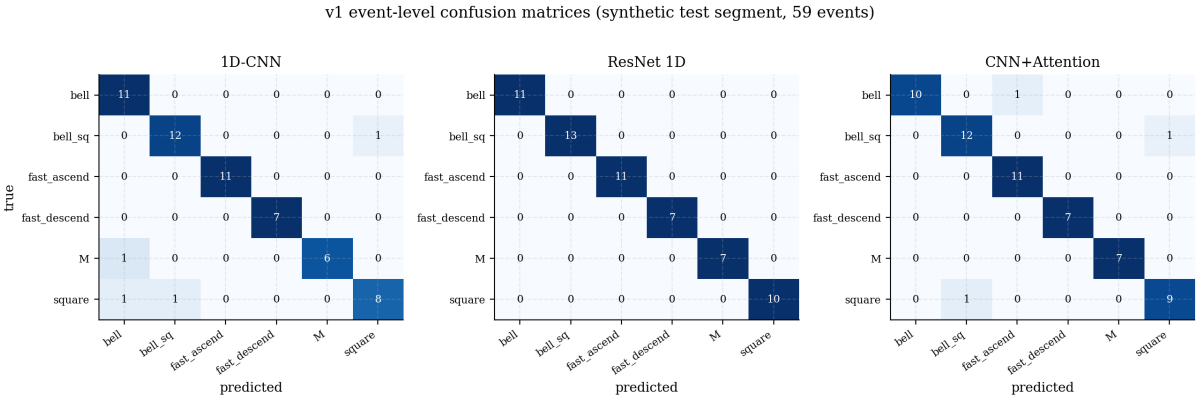

Figure 5.2. Event-level confusion matrices on the 59-event synthetic segment. The ResNet is on the diagonal; the misses of the other two models fall on the visually similar morphologies.

Model	Hits	Misses	False alarms	Macro P / R	Macro F_1
1D-CNN	55	4	0	0.943 / 0.930	0.934
ResNet 1D	59	0	0	1.000 / 1.000	**1.000**
CNN+Attention	56	3	0	0.957 / 0.955	0.955

Table 5.2. Event-level results on the synthetic segment (59 events).

5.3 Robustness to distribution shift

A deployed beacon will see conditions that differ from training. We stress each model along two independent axes by regenerating small test sets (50 events each) while varying one factor (fig. 5.3): the background-noise level (the noise std is multiplied by $m_{\text{noise}} \in [1, 3]$, and the anomaly amplitude is scaled with it, so the test is one of *noise texture and normalisation*, not raw SNR) and the shape-deformation level (ν multiplied by $m_{\text{deform}} \in [1, 8]$). All models start at $\sim 98\%$ F_1; table 5.3 reports where each falls below 50%.

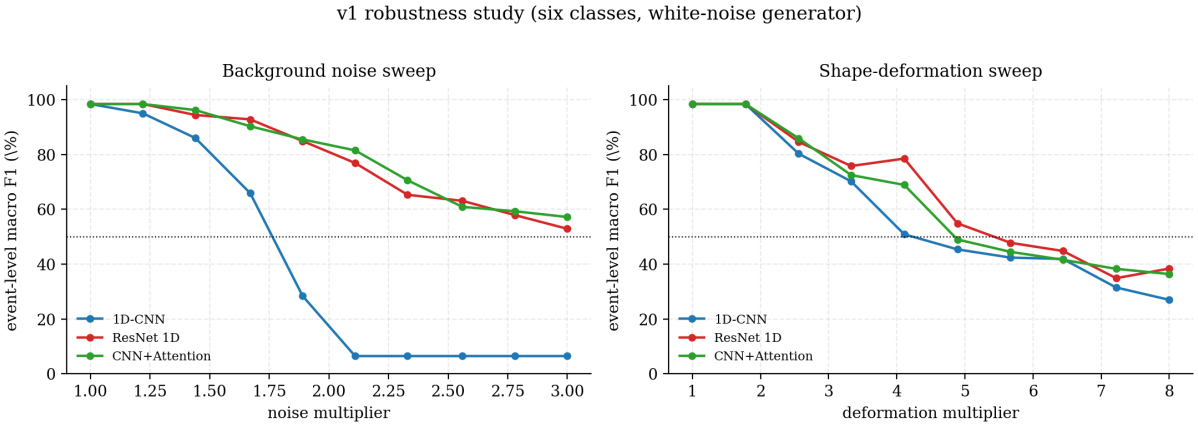

Figure 5.3. Robustness. Event-level macro F_1 vs background-noise multiplier (left) and vs shape-deformation multiplier (right).

Model	Noise: $F_1 < 50\%$ at	Deformation: $F_1 < 50\%$ at
1D-CNN	$m_{\text{noise}} \approx 1.9$ (collapses to $\sim 6\%$)	$m_{\text{deform}} \approx 4.9$
ResNet 1D	never ($\geq 53\%$ up to $3\times$)	$m_{\text{deform}} \approx 5.7$
CNN+Attention	never ($\geq 57\%$ up to $3\times$)	$m_{\text{deform}} \approx 4.9$

Table 5.3. Robustness thresholds. The 1D-CNN collapses to a single class under moderate noise growth; the two group-norm models do not.

The contrast on the noise axis is stark: the baseline 1D-CNN drops below 50% by $m_{\text{noise}} \approx 1.9$ and then collapses to a single-class $\sim 6\%$ plateau, whereas the ResNet and the CNN+Attention stay above 50% across the whole sweep. The two robust models share group normalisation and skip connections; the baseline's batch-norm running statistics, fitted at the training noise level, become invalid as the input distribution shifts. On the shape-deformation axis all three degrade gracefully, the ResNet most slowly.

5.4 Computational cost

Table 5.4 collects the cost–performance trade-off. The 1D-CNN is by far the smallest and trains in a third of the time, but pays for it in accuracy and, above all, noise robustness. The ResNet trains in about ten minutes on a laptop GPU and gives the best accuracy, the best robustness and a perfect event-level score, for a third of the parameters of the CNN+Attention. Inference is fast for all three (thousands of windows per second); the throughput differences are small and partly reflect first-run GPU warm-up for the baseline.

Model	Parameters	Train time	Inference (win/s)	Event F_1
1D-CNN	22,727	$\sim$ 4.3 min	$\sim$ 10,700	0.934
ResNet 1D	74,631	$\sim$ 10.0 min	$\sim$ 14,200	**1.000**
CNN+Attention	109,655	$\sim$ 10.2 min	$\sim$ 15,200	0.955

Table 5.4. Cost vs performance (single RTX 5060 Laptop GPU).

5.5 Discussion

Three findings stand out. First, **residual learning with group normalisation is the single biggest win**: at equal depth and an identical head, the ResNet beats the plain 1D-CNN on every axis, and the gap explodes under noise growth. Second, **self-attention buys no extra robustness here**: the CNN+Attention sits between the other two, close to the ResNet but below it. The $W = 512$ window ($\leq$ 800-sample events) is short enough that a convolutional receptive field already covers an event, leaving little long-range context for attention to exploit — we would expect attention to pay off on much longer windows. Third, the **cheapest model is the most fragile**: the 1D-CNN's single-class collapse under noise makes it unsuitable for a beacon that runs across seasons. Overall the ResNet 1D is the best trade-off of accuracy, robustness and cost on this benchmark.

Chapter 6

Conclusion and perspectives

6.1 Summary

We formalised the identification of radiological events as sliding-window multi-class classification of a noisy gamma-dose signal, and validated a deep-learning approach to it on a controlled synthetic benchmark. The benchmark rests on a deliberately simplified signal model — white Gaussian noise on an Ornstein–Uhlenbeck baseline, with parameters calibrated to four months of real 2015 recordings and six injected anomaly morphologies. On held-out synthetic data, all three architectures reach a strong event-level performance (macro F_1 from 0.93 to 1.00 on a 59-event segment), with no false alarms. The residual network is the best overall: a perfect event-level score, the highest window accuracy, and — decisively — the only model besides the attention hybrid that resists background-noise growth, while the baseline 1D-CNN collapses to a single class. Self-attention did not improve on the ResNet at the 512-sample window scale.

6.2 Architecture recommendation

For this task we recommend the **ResNet 1D**: it combines the best accuracy and robustness with a moderate cost ($\sim$ 75k parameters, $\sim$ 10 min to train on a laptop GPU). The 1D-CNN is attractive only where compute is extremely constrained *and* noise conditions are stable; the CNN+Attention is the natural candidate if the window is later extended to capture long-range context.

6.3 Limitations

The results are, by construction, *synthetic*. The signal model is simple in two ways that matter: its high-frequency noise is independent in time (real sensor noise is correlated), and its anomaly density is far higher than a real beacon's. As stressed throughout, this is appropriate for validating the approach but means the numbers above should not be read as real-world detection rates. There is also no labelled real test set, so real performance cannot yet be measured.

6.4 Perspectives

The natural next step — in the supervisor's words, "to go further" — is to close the gap to real signals and only then evaluate on them:

- **More realistic signal models.** Replace the white noise by a coloured (e.g. autoregressive)

process matching the empirical autocorrelation of the real residual, lower the anomaly density towards the real one, and calibrate amplitudes/durations and the morphology set to events actually observed. The very same validation pipeline can then be re-run on this harder benchmark.

- **A labelled real test set.** Expert labelling of the real recordings would, for the first time, allow event-level precision and recall to be measured on real data — the only sound way to evaluate networks trained on a synthetic model.
- **Explainability.** The convolutional models expose their last feature map for Grad-CAM [13], and the encoder's attention maps are directly interpretable; both would let an operator verify that a prediction attends to the right part of an event.
- **Longer windows and richer inputs.** Longer windows would give the attention model the context it lacked here, and the convolutions already accept extra channels, so the companion meteorological signals (temperature, humidity, pressure) could be added as inputs.

6.5 Final word

On a clean, controlled benchmark the approach works well and the architectural comparison is informative: depth and group normalisation, not attention, are what buy robustness at this window length. The honest limitation — that this is synthetic validation, not real-world evaluation — is also the clearest signpost for the next iteration.

Bibliography

[1] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E. Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016.

[2] Shaojie Bai, J. Zico Kolter, and Vladlen Koltun. An empirical evaluation of generic convolutional and recurrent networks for sequence modeling. *arXiv preprint arXiv:1803.01271*, 2018.

[3] Ane Blázquez-García, Angel Conde, Usue Mori, and Jose A. Lozano. A review on outlier/anomaly detection in time series data. *ACM Computing Surveys*, 54(3):1–33, 2021.

[4] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of NAACL-HLT*, pages 4171–4186, 2019.

[5] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep residual learning for image recognition. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pages 770–778, 2016.

[6] Zhangfang Hu and Libujie Chen. EEG-based emotion recognition using convolutional recurrent neural network with multi-head self-attention. *Applied Sciences*, 12(21):11255, 2022.

[7] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *International Conference on Machine Learning (ICML)*, pages 448–456, 2015.

[8] Hassan Ismail Fawaz, Germain Forestier, Jonathan Weber, Lhassane Idoumghar, and Pierre-Alain Muller. Deep learning for time series classification: a review. *Data Mining and Knowledge Discovery*, 33(4):917–963, 2019.

[9] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization. In *International Conference on Learning Representations (ICLR)*, 2015.

[10] Elisabeth Lahalle. Suivi dynamique de pollution radiologique par IA. Project brief, Pôle Projet Data Science, CentraleSupélec / L2S, 2025.

[11] Yann LeCun, Léon Bottou, Yoshua Bengio, and Patrick Haffner. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11):2278–2324, 1998.

[12] Adam Paszke, Sam Gross, Francisco Massa, et al. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 8024–8035, 2019.

[13] Ramprasaath R. Selvaraju, Michael Cogswell, Abhishek Das, Ramakrishna Vedantam, Devi Parikh, and Dhruv Batra. Grad-CAM: Visual explanations from deep networks via gradient-based localization. In *IEEE International Conference on Computer Vision (ICCV)*, pages 618–626, 2017.

[14] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems (NeurIPS)*, pages 5998–6008, 2017.

[15] Zhiguang Wang, Weizhong Yan, and Tim Oates. Time series classification from scratch with deep neural networks: A strong baseline. In *International Joint Conference on Neural Networks (IJCNN)*, pages 1578–1585, 2017.

[16] Yuxin Wu and Kaiming He. Group normalization. In *European Conference on Computer Vision (ECCV)*, pages 3–19, 2018.

Appendix A

Appendices

A.1 Generator configuration

Parameter	Value
Total length N	10^7 samples (written in $5 \cdot 10^4$ chunks)
Number of anomalies	5,000–8,000 ($\sim$ 6–8 / 10k samples)
Injection jitter	$\pm$200 samples
Anomaly amplitude	$\sigma \cdot \mathcal{U}(2.5, 7.6)$ (SNR units)
Anomaly duration	$\mathcal{U}[200, 800]$ samples
Deformation level ν	$\mathcal{U}(0.02, 0.10)$
HF noise	i.i.d. $\mathcal{N}(0, \sigma^2)$, $\sigma = 2.53$
Baseline (OU)	$\mu = 99.26$, $\theta = 4 \cdot 10^{-6}$, $\sigma_{\mathrm{OU}} = 0.0373$
Templates / classes	6 morphologies + background ($K = 7$)

Table A.1. Synthetic generator configuration.

A.2 Training and inference configuration

Hyperparameter	Value
Window size W	512
Stride (train / inference)	32 / 10
Labelling threshold τ_{cov}	0.10 of the window's samples (majority anomaly class)
Normalisation	per-window centring, global $\sigma_{\mathrm{train}} = 4.21$
Split	temporal 80/20, then random 80/20 on train
Optimiser	Adam, β_1=0.9, β_2=0.999, ε=10^{-8}
Learning rate / batch	10^{-3} / 128
Max epochs	35
LR scheduler	`ReduceLROnPlateau`(patience 2, factor 0.5)
Early stopping	patience 10, min-delta $5 \cdot 10^{-3}$
Loss	class-weighted cross-entropy, eq. (1.2)
Class weights $\mathbf{w}$	$(0.46, 1.18, 1.27, 1.32, 1.24, 1.15, 1.31)$
Event extraction	background-prob. threshold 0.3, bridge gaps ≤ 50
Probability smoothing	30-sample rolling mean

Table A.2. Shared training, inference and event-extraction settings.

A.3 Anomaly-template format

Each template is a CSV of keypoints with columns `time`, `value` (both normalised to $[0, 1]$), `t_minus`, `t_plus`, `v_minus`, `v_plus` (flags allowing jitter in each direction), and `interp` (segment interpolation: `linear`, `exp_up`, `exp_down` or `bell`). The deformation perturbs the free keypoints by a Gaussian of std ν under these direction constraints, enforces time monotonicity, and interpolates to a dense curve rescaled to the target amplitude and duration.

A.4 Software environment

Experiments ran on Windows 11 with Python 3.13 and PyTorch 2.12 (CUDA 12.8) on a single NVIDIA RTX 5060 Laptop GPU, using NumPy, Pandas, Matplotlib, SciPy and scikit-learn for data handling, metrics and figures. The repository contains the generator (`data-gen/`), the models and training/evaluation scripts (`architectures/`), and this report with a figure-rebuild script (`documentation/`).